

同濟大學

TONGJI UNIVERSITY

《计算机系统实验》

实验报告

实验名称

CPU 改造实验

实验成员

曾崇然 (2152809)

日期

二零二四年 4 月 12 日

### 1、实验目的

- 回顾 MIPS 五阶段流水线 CPU 的基本结构
- 学习教学用 CPU 各模块的编写以及模块之间的连接结构
- 加深对 Verilog 语言的理解以及对硬件开发的理解
- 为之后一致操作系统的实验做准备工作

### 2、实验内容

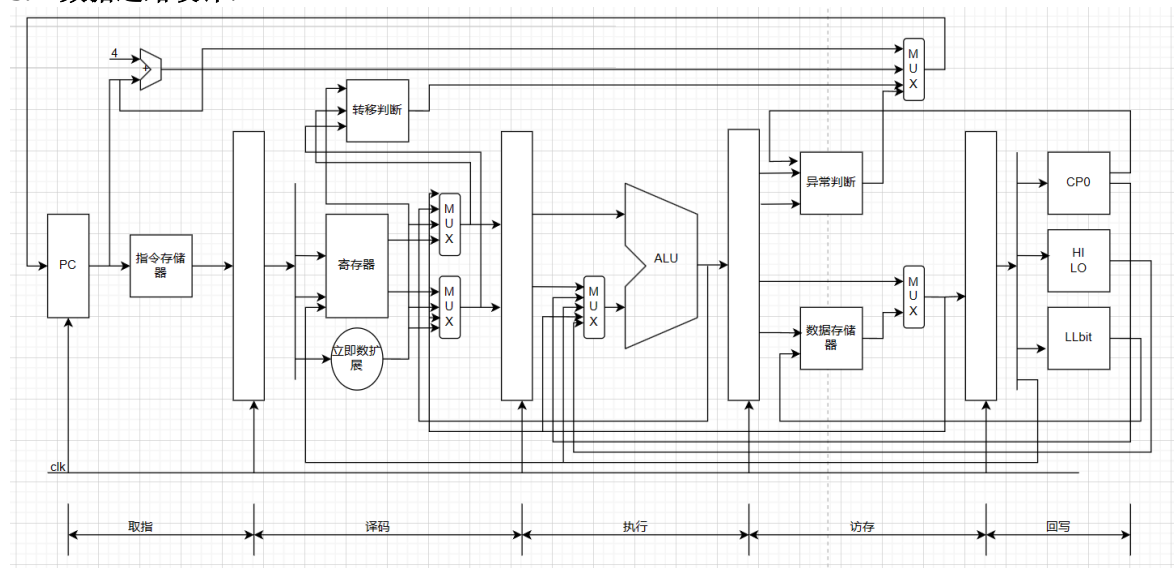
参考《自己动手写 CPU》改造 54 条指令的 MIPS 流水线 CPU，支持到 89 条指令，实现 CP0、异常处理，需要新增的 35 条指令如下：

- 移动操作指令：  
movn、movz
- 算术操作指令：  
clo、madd、maddu、msub、msubu
- 转移指令：  
b、bal、bgezal、bgtz、blez、bltz、bltzal
- 加载存储指令：  
ll、lwl、lwr、sc、swl、swr
- 异常相关指令：  
tge、tgeu、tlr、tlru、tne、teqi、tgei、tgeiu、tlr、tlriu、tnei
- 其他指令：  
nop、ssnop、sync、pref

本次实验中实现的 MIPS CPU 为五级流水线 CPU，分别有取指、译码、执行、访存、回写五个阶段。采用大端模式。采用哈佛结构，有分开的指令存储器和数据存储器。

### 3、实验步骤和结果

#### 3.1 数据通路设计：



#### 3.2 功能模块的定义实现：

- PC 寄存器：保存当前取指的地址
- IF\_ID 流水寄存器：保存取指阶段送到译码阶段的数据

- 译码阶段模块：组合逻辑电路，负责进行指令的译码
  - 通用寄存器堆：通用寄存器堆，负责暂存数
  - ID\_EX 流水寄存器：保存译码阶段传递到执行阶段的数据
  - 执行阶段模块：组合逻辑电路，负责进行指令的执行，包括计算等
  - EX\_MEM 流水寄存器：保存执行阶段传递到访存阶段的数据
  - 访存阶段模块：组合逻辑电路，负责访问数据存储器，同时异常的处理也在这个模块进行
  - HILO 寄存器：存储一些特定指令的结果，如乘法，除法，mtlo 等
  - 控制器模块：产生控制信号，控制处理器的暂停和异常处理等
  - 除法模块：除法指令的执行模块
  - Llbit 寄存器：用于实现保证操作原子性的两条指令的寄存器
  - CP0 协处理器模块：0 号协处理器模块，负责存储异常相关的数据和进行异常的处理
  - 指令存储器模块：rom，存储要执行的指令序列
  - 数据存储器模块：ram，存储执行程序所需的数据
  - 处理器模块：负责连接 CPU 内部的各个部件
  - 顶层模块：负责连接 CPU 和数据存储器以及指令存储器，形成完整的处理器
- 对应代码放于压缩包中

### 3.3 编写测试文件进行仿真：

#### 1) 编写测试文件

```

) module openmips_min_sopc_tb();

    reg CLOCK_50;
    reg rst;

    // 每隔10ns, CLOCK_50信号翻转依次, 所以一个周期是20ns, 对应50MHz
)    initial
)    begin
        CLOCK_50 = 1'b0;
        forever #10 CLOCK_50 = ~CLOCK_50;
)    end

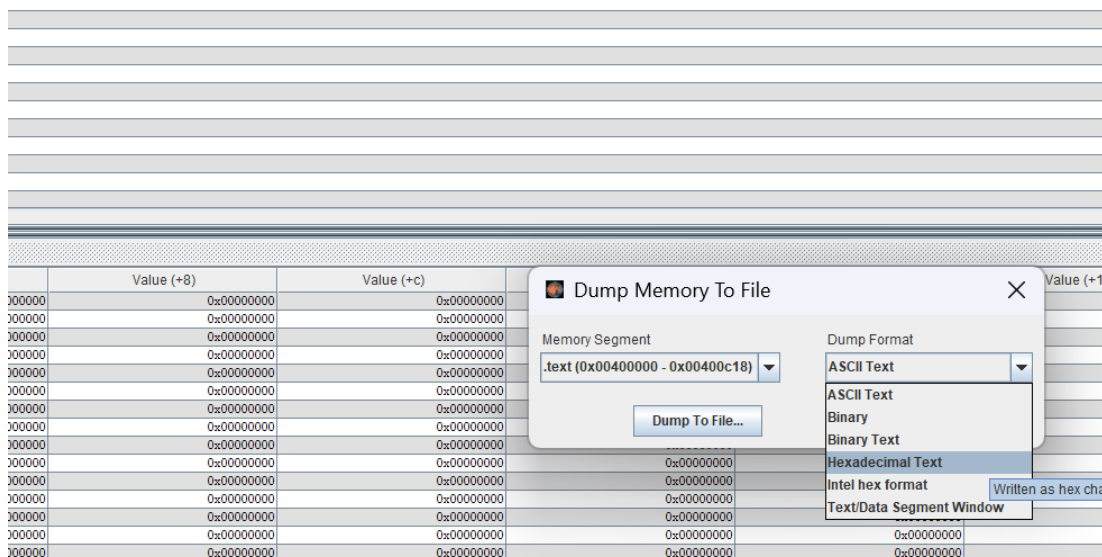
)    // 最初时刻, 复位信号有效, 在第195ns, 复位信号无效, 最小SOPC开始运行
)    initial
)    begin
        rst = `RstEnable;
        #195 rst = `RstDisable;
)    end

    // 例化最小SOPC
    openmips_min_sopc openmips_min_sopc0(
        .clk(CLOCK_50),
        .rst(rst)
    );

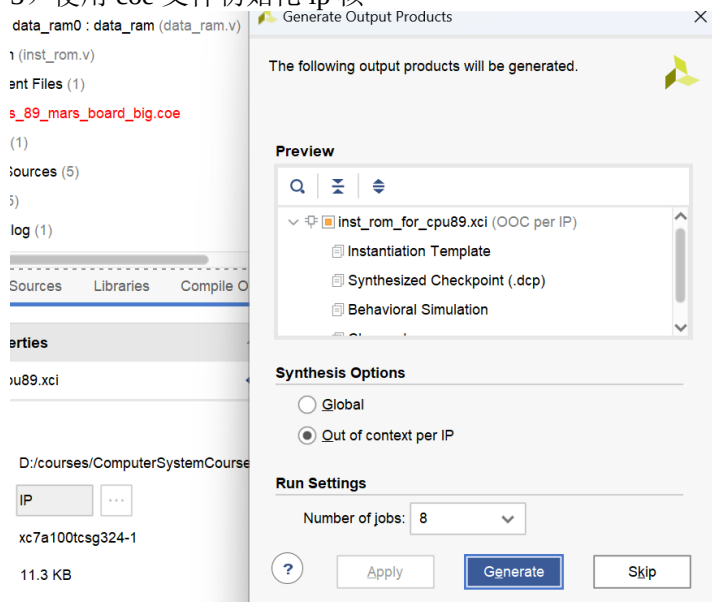
) endmodule

```

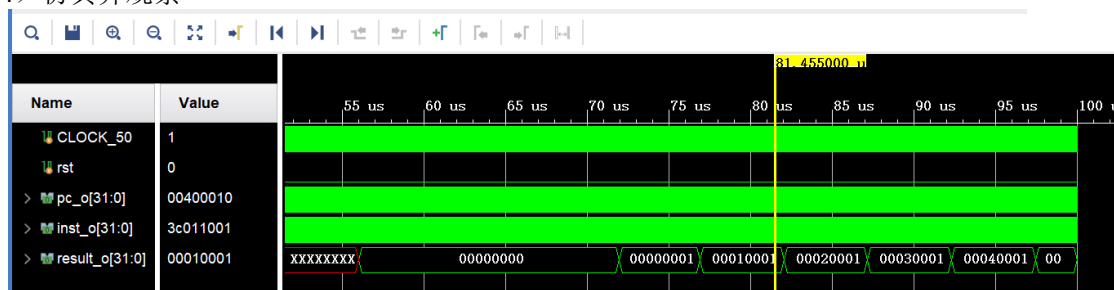
#### 2) 使用 Mars 将测试文件转为 coe 文件



### 3) 使用 coe 文件初始化 ip 核



### 4) 仿真并观察



result 值记录了数据存储单元中第一个存储单元的值，其低位为 0001，表明执行结果正确，其高位不断增加，表明时钟中断正常产生。

### 3.4 下板验证

#### 1) 添加下板相关的模块

- 分频器：用于降低时钟频率，为数码管和处理器提供对应的时钟信号
- 7 段数码管：用于显示输出的结果
- 下板的顶层模块，用于连接分频器，处理器和数码管

```

module openmips_board(
    input clk_in,        // 输入时钟
    input reset,         // 复位信号
    input en_clk,        // 时钟选择
    input [1:0] choose,  // 显示选择
    output [7:0] o_seg,
    output [7:0] o_sel
);

    wire clk_cpu;
    wire clk_seg;
    wire [31:0] i_data;
    wire [31:0] inst, pc, result;

    ////////////////////////////////////////////
    // 选择显示 inst, pc, result
    divider#(4) div_seg (clk_in, reset, clk_seg);
    divider#(100000) div_cpu (clk_in, reset, clk_cpu);
    seg7x16 seg7 (clk_seg, reset, i_data, o_seg, o_sel);
    assign i_data = (choose[1]) ? inst : (choose[0] ? pc : result);

    ////////////////////////////////////////////
    // pcpu_onboard 顶层部分
    wire clk_real;
    assign clk_real = en_clk ? clk_seg : clk_cpu;
    openmips_min_sopc docpu_top(clk_real, reset, pc, inst, result);

endmodule

```

## 2) 添加约束文件

```

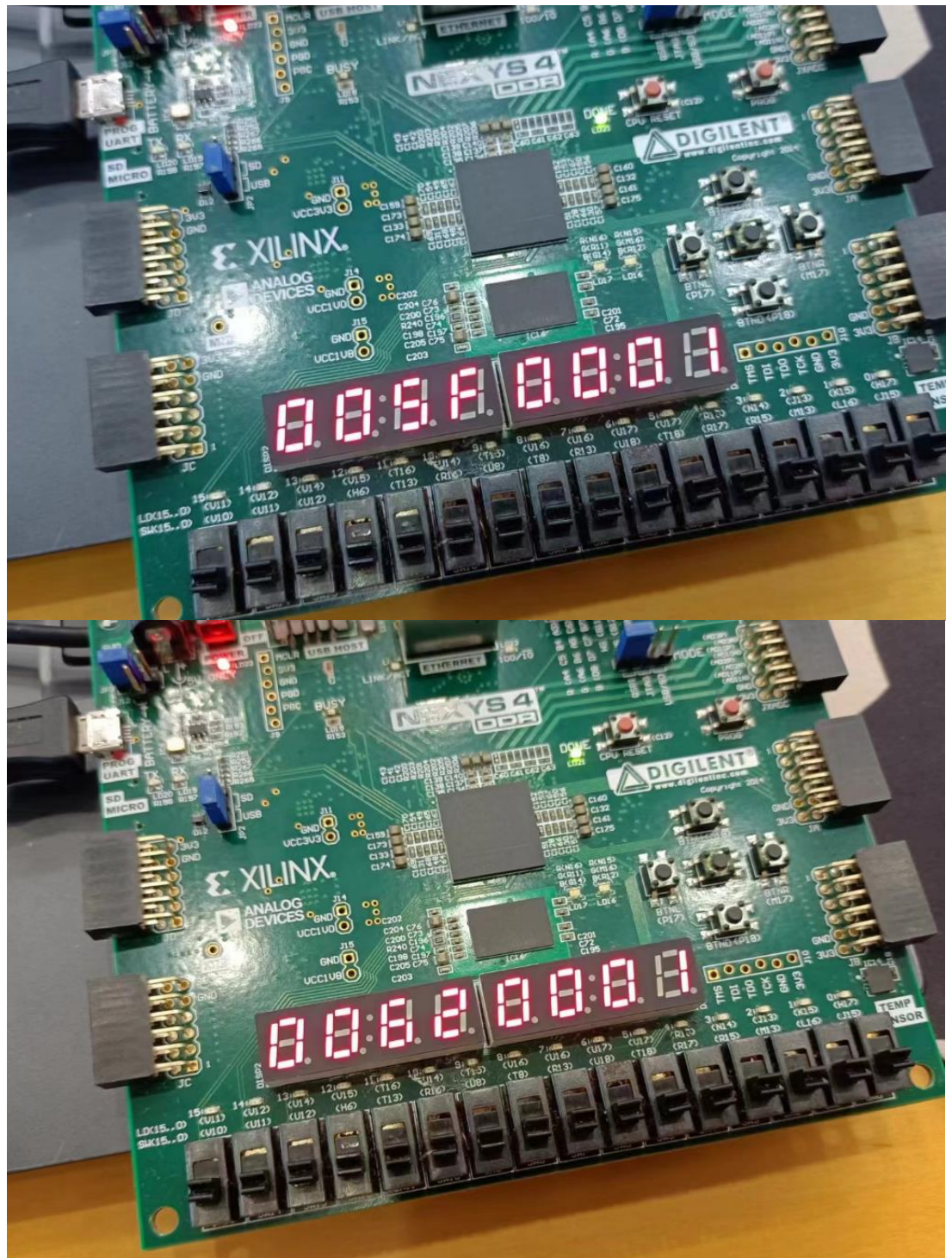
1 create_clock -period 100.000 -name clk_pin -waveform {0.000 50.000} [get_ports clk_in]
2 set_input_delay -clock [get_clocks *] 1.000 [get_ports reset]
3 set_output_delay -clock [get_clocks *] 0.000 [get_ports -filter { NAME =~ "*" && DIRECTION == "OUT" }]
4
5 set_property PACKAGE_PIN E3 [get_ports clk_in]
6 set_property PACKAGE_PIN N17 [get_ports reset]
7 set_property PACKAGE_PIN V10 [get_ports en_clk]
8
9 set_property PACKAGE_PIN T10 [get_ports {o_seg[0]}]
10

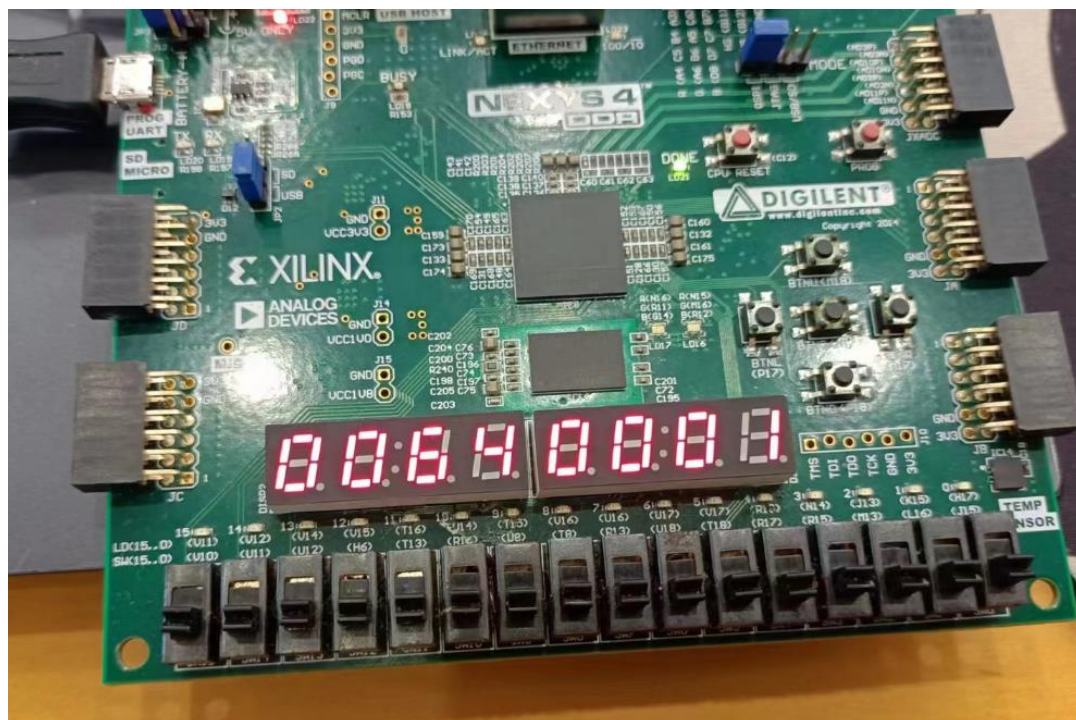
```

## 3) 生成 bit 流下板并观察现象

可以看到数码管的低 4 位显示为 0001，表明指令执行的结果正确，高 4 位不断增加，表明时钟中断的次数不断增加，下板结果正确

装  
订  
线





#### 4、实验总结

在本次实验中，我参考《自己动手写 CPU》一步一步实现了支持 89 条指令的 MIPS 五级流水线 CPU。

- 在这个过程中，我对指令的流水执行以及 MIPS 五级流水线 CPU 每个阶段的完成的功能有了更加深刻的理解，能够建立基本的流水结构；
- 对 Verilog 语言的使用更加熟练、使用 Vivado 进行开发、仿真、调试和下板的水平得到了提升，学会了使用虚拟机搭建 Linux 环境，使用相应的 GNU 对 MIPS 指令进行编译；
- 了解了指令流水执行中可能产生的冲突问题，并掌握了其基本的解决方法，包括数据前推和延迟操作技术等；
- 了解了乘除法和其他指令执行时长不同，如何在不增长时钟周期的情况下完成这类指令的方法；
- 了解了如何通过 ll 和 sc 指令保证操作的原子性
- 了解了如何使用协处理器 CP0 对程序执行过程中产生的异常进行处理；